Mahmoud Abou-Zaeid

SOFTWARE REQUIREMENTS SPECIFICATION

DevPilot AI

AI-Powered Developer Problem Solving, Knowledge, Practice and Code Review Platform

PROJECT NAME	**DevPilot AI**
DOCUMENT VERSION	1.0
DOCUMENT TYPE	Software Requirements Specification (SRS)
PROJECT TYPE	AI-Powered Developer Problem Solving, Knowledge, Practice and Code Review Platform
ISSUE DATE	2 October 2026
PREPARED BY	Mahmoud Abou-Zaeid

Table of Contents

1. Project Overview 3
2. Project Objectives 3
3. Target Users 4
4. Core System Workflow 5
5. Functional Requirements 6
5.1 Developer Account 6
5.2 AI Debugging Workspace 6
5.3 Solution Feedback Loop 7
5.4 Debugging History 7
5.5 Problem Identification 7
5.6 Public Knowledge Base 8
5.7 Problem Statistics 8
5.8 Similarity Search 9
5.9 Token Optimization 9
5.10 Solution Evolution 9
5.11 Educational Article Generation 10
5.12 Article Versioning 10
5.13 Developer Practice Challenges 10
5.14 GitHub Integration 11
5.15 GitHub-Based Debugging 11
5.16 AI Code Review 12
5.17 Challenge GitHub Repository 13
5.18 Challenge Submission and Evaluation 13
5.19 Knowledge Base Challenge Integration 14
5.20 Telegram Bot 14
5.21 Discord Integration 14
5.22 Cloudflare Turnstile 15
5.23 AI Gateway 15
5.24 Free LLM Provider Failover 15
5.25 AI Prompt Management 16
6. Administrator Requirements 16
7. Non-Functional Requirements 17
7.1 Performance 17
7.2 Security 17
7.3 Privacy 17
7.4 Reliability 17
7.5 Maintainability 18
8. Proposed Backend Architecture 18

9. Proposed Database Entities 20
10. Important Data Relationships 20
11. API Modules 21
12. AI Processing Pipeline 22
13. Token Optimization Strategy 22
14. Knowledge Quality Strategy 23
15. Search System 23
16. Notifications 23
17. Logging and Auditing 24
18. Error Handling 24
19. MVP Scope 25
20. Out of Scope for Initial Version 25
21. Deployment 26
22. Security Principles for GitHub 26
23. Future Enhancements 26
24. Project Differentiator 27
25. Success Criteria 28

1. Project Overview

DevPilot AI is a web-based platform designed exclusively for software developers.

The platform helps developers solve real-world programming problems through AI-powered debugging. The system analyzes the developer's problem, error, code, logs, or screenshots, explains the root cause, and provides a step-by-step solution.

After every solution attempt, the developer can select:

1. Solved
2. Still Have a Problem

If the problem is not solved, the AI performs another debugging attempt using the new information and provides another solution. This loop continues until the developer confirms that the problem is solved.

When a problem is solved, the system stores the complete problem and solution history in the database and uses the information to generate an educational article.

The platform also builds a public developer knowledge base similar in concept to Stack Overflow, where developers can search and read previously solved problems.

The system continuously improves its knowledge by collecting successful solutions from different developers, identifying similar problems, and synthesizing better solutions and updated educational articles.

DevPilot AI also integrates with GitHub to support repository debugging, AI code review, challenge repository creation, and practical developer training.

2. Project Objectives

The main objectives are:

- Help developers solve programming problems faster.
- Provide understandable explanations instead of only giving final answers.
- Guide developers through debugging step by step.
- Reuse previously successful solutions.
- Reduce unnecessary LLM API requests and token consumption.
- Build a continuously improving developer knowledge base.
- Convert solved real-world problems into educational articles.
- Allow developers to practice problems through practical coding challenges.
- Integrate GitHub into the debugging and learning workflow.
- Provide AI-powered code review.
- Provide a Telegram interface for searching existing solutions.
- Support multiple free LLM providers with automatic failover.
- Build a scalable architecture while respecting free hosting and API limitations.

3. Target Users

PRIMARY USER

Software Developer

SECONDARY USER

Platform Administrator

The platform is intended specifically for developers and technical users.

4. Core System Workflow

The main workflow takes a developer problem through analysis, similarity search, AI debugging, and feedback. Solved problems become knowledge, articles, practice challenges, and improvements to the knowledge base itself.

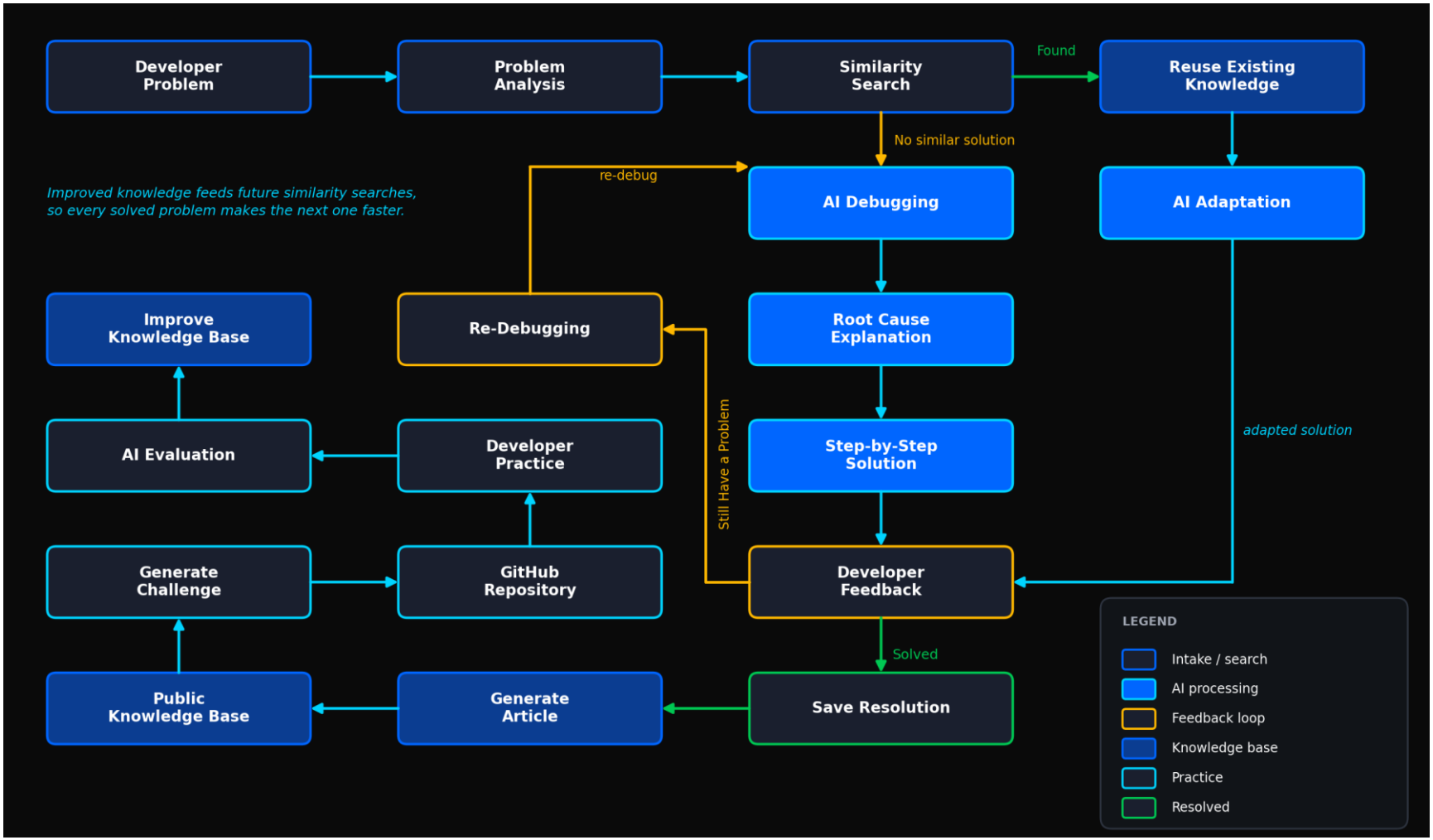


Figure 1. Core system workflow — blue: AI processing, amber: feedback loop, green: resolved path.

5. Functional Requirements

5.1 Developer Account

The system shall allow developers to:

- Register an account.
- Log in.
- Log out.
- Manage their profile.
- View their previous problems.
- View their debugging history.
- View their challenges.
- View challenge evaluation results.
- Connect their GitHub account.
- Manage connected integrations.

The system should support role-based authorization.

Initial roles:

- Developer
- Administrator

5.2 AI Debugging Workspace

The system shall provide a dedicated workspace for submitting technical problems.

A developer may provide:

- Problem description.
- Error message.
- Exception.
- Stack trace.
- Source code.
- Logs.
- Screenshots.
- Relevant technical information.
- Technology/framework information.

The system shall create a Problem record for each new debugging session.

The AI shall:

- Analyze the submitted information.
- Identify possible causes.
- Determine the probable root cause.
- Explain the problem.
- Provide a step-by-step solution.
- Explain why each step is required.
- Provide code examples when appropriate.

- Avoid presenting unsupported assumptions as confirmed facts.

The solution shall be stored as a debugging attempt.

5.3 Solution Feedback Loop

After each AI solution, the system shall ask the developer:

> **“Did this solve your problem?”**

Available options:

- Solved
- Still Have a Problem

Solved

- Mark the problem as solved.
- Store the successful solution.
- Store the complete debugging history.
- Generate or update the related educational article.
- Update problem statistics.
- Make the successful knowledge available for future similarity searches.

Still Have a Problem

- Request additional information when necessary.
- Store the developer’s feedback.
- Start another debugging attempt.
- Provide a new or modified solution.
- Repeat the feedback cycle.

5.4 Debugging History

Each problem shall maintain all debugging attempts.

Example:

Problem	Attempt	AI Analysis	Solution	Developer Feedback
Problem	Attempt 1	AI Analysis	Solution	Developer Feedback
Problem	Attempt 2	AI Analysis	Solution	Developer Feedback
Problem	Attempt 3	AI Analysis	Solution	**Solved**

The system shall never lose previous attempts.

The history shall be available for analysis and knowledge generation.

5.5 Problem Identification

Every solved knowledge problem shall receive a unique Problem Code.

Example:

`DEV-000001`	`DEV-000002`	`DEV-000003`

The public problem listing shall display:

- Problem title.
- Problem code.
- Short description.

- Technologies/tags.
- Number of developers who faced the problem.
- Relevant statistics.

5.6 Public Knowledge Base

The system shall provide a public knowledge base for developers.

The knowledge base shall be similar in concept to a developer Q&A/knowledge platform.

Users shall be able to:

- Browse problems.
- Search problems.
- Filter problems.
- Open problem articles.
- View solutions.
- View explanations.
- View related challenges.

Each article shall contain:

- Problem title.
- Problem code.
- Problem description.
- Error or code context.
- Root cause.
- Explanation.
- Step-by-step solution.
- Explanation of each step.
- Technologies/tags.
- Problem statistics.
- Related problems.
- Practice challenge when available.

5.7 Problem Statistics

The system shall track statistics for problems.

Possible statistics include:

- Number of developers who reported the problem.
- Number of successful resolutions.
- Number of failed attempts.
- Number of debugging attempts.
- Number of successful solution variants.
- Technologies associated with the problem.
- Challenge participation count.

Statistics shall be used to improve the knowledge base and solution generation.

5.8 Similarity Search

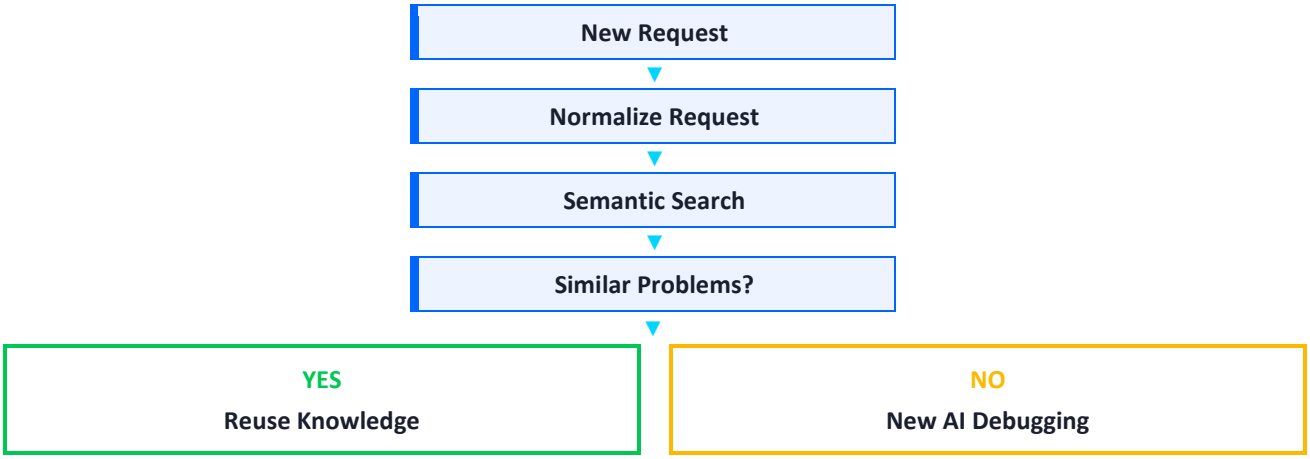
Before sending every new problem directly to an LLM, the system shall search the existing knowledge base for similar problems.

The similarity process should consider:

- Problem description.
- Error messages.
- Stack traces.
- Code context.
- Technologies.
- Frameworks.
- Tags.
- Previously successful solutions.

The system should use semantic similarity/embeddings where technically feasible.

Possible flow:



5.9 Token Optimization

The platform shall minimize unnecessary LLM usage.

If a highly similar solved problem exists:

- Retrieve the previous request.
- Retrieve the successful solution.
- Provide the previous knowledge as context to the LLM.
- Ask the LLM to adapt the existing solution to the current case.

The system shall not blindly reuse an old solution without validation/adaptation.

Every new successful adaptation shall be stored.

> This creates a growing repository of proven solutions.

5.10 Solution Evolution

When multiple developers solve similar problems using different successful solutions, the system shall be able to collect the successful solution variants.

Example:

Request	Solution	Outcome
Request A	Solution A	Success
Request B	Solution B	Success
Request C	Solution C	Success

The AI may synthesize these successful solutions into an improved solution.

The improved solution can be used to update the related knowledge article.

The system should preserve historical versions of the article/solution.

The knowledge base is therefore intended to be continuously improving.

5.11 Educational Article Generation

When a problem is successfully solved, the system shall generate an educational article.

The article should include:

1. Problem
2. Error/Code
3. Why the problem happens
4. Root cause
5. Step-by-step solution
6. Explanation of each step
7. Best practices
8. Prevention recommendations
9. Related problems
10. Practice challenge

The article should be understandable to another developer who has never seen the original problem.

5.12 Article Versioning

The system shall support article updates.

When new successful solutions provide better information:

- Generate a new article version.
- Preserve the previous version.
- Store the reason for the update.
- Keep the latest approved/generated solution as the current version.

The system should allow administrators to review generated knowledge when required.

5.13 Developer Practice Challenges

After a problem is solved, the system shall be able to generate a small coding challenge based on the same concept.

The challenge should test whether the developer understood the problem.

The challenge may contain:

- Title.
- Description.
- Requirements.
- Constraints.
- Expected behavior.
- Starter code.
- Acceptance criteria.
- Difficulty.
- Technology.
- Evaluation criteria.

The challenge shall not expose the complete solution to the learner.

5.14 GitHub Integration

The platform shall support GitHub integration.

A developer shall be able to connect a GitHub account.

The integration may support:

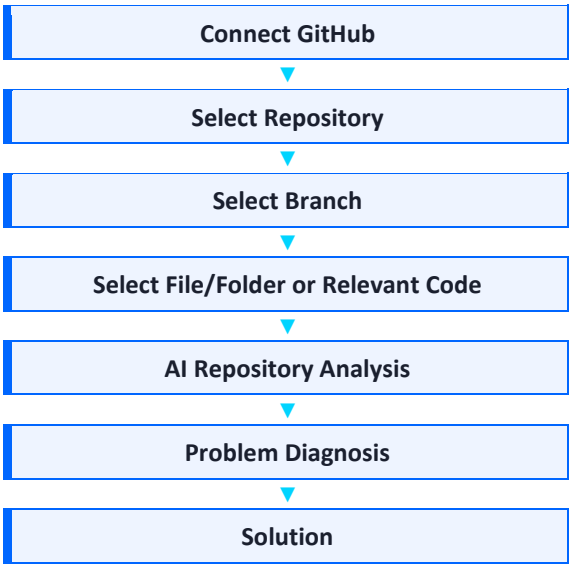
- OAuth authentication.
- Repository access.
- Repository listing.
- Branch selection.
- File retrieval.
- Commit information.
- Pull request information.
- Webhooks.
- Repository creation for challenges.

GitHub access shall follow the minimum permissions required by each feature.

5.15 GitHub-Based Debugging

A developer shall be able to connect a GitHub repository and identify where the problem exists.

Possible workflow:



The AI should identify relevant files and explain relationships between files when enough repository context is available.

The system shall avoid sending unnecessary repository files to the LLM to reduce token usage.

5.16 AI Code Review

The platform shall provide a Code Review feature.

The developer may select:

- Repository.
- Branch.
- Pull Request.
- Specific files.
- Relevant code.

The AI shall analyze the selected code.

The report may include:

- Bugs.
- Security issues.
- Performance issues.
- Code smells.
- SOLID violations.
- Maintainability problems.
- Architecture concerns.
- Best-practice recommendations.

The report should identify:

- File.
- Relevant code section.
- Problem.
- Severity.
- Explanation.

- Suggested improvement.

The review should produce an organized report.

5.17 Challenge GitHub Repository

When a developer starts a challenge:

- The system shall create a dedicated GitHub repository when permitted.
- The repository shall contain the challenge description.
- The repository may contain starter code.
- The README shall explain requirements and expected behavior.

Example:

`DevPilot-Challenge-JWT-Authentication`

The challenge repository shall be associated with:

- Developer.
- Challenge.
- Problem.
- Repository.
- Attempt.

5.18 Challenge Submission and Evaluation

The developer shall implement the challenge and push the solution to GitHub.

The platform shall evaluate the submitted implementation.

Evaluation may consider:

- Functional correctness.
- Requirement completion.
- Code quality.
- Security.
- Performance.
- Architecture.
- Understanding of the original problem.
- Best practices.

The system shall generate an evaluation report.

Example — Challenge Result:

Criterion	Rating
Understanding	**Good**
Implementation	**Excellent**
Security	**Needs Improvement**
Code Quality	**Good**

The system shall provide constructive feedback.

5.19 Knowledge Base Challenge Integration

Each relevant article may contain:

PRACTICE CHALLENGE

[Start Challenge]

When another developer starts the challenge:

1. A challenge attempt is created.
2. A GitHub repository can be created.
3. The developer completes the task.
4. The system evaluates the solution.
5. The result is associated with the challenge attempt.

This transforms the knowledge base from a static documentation system into an interactive learning environment.

5.20 Telegram Bot

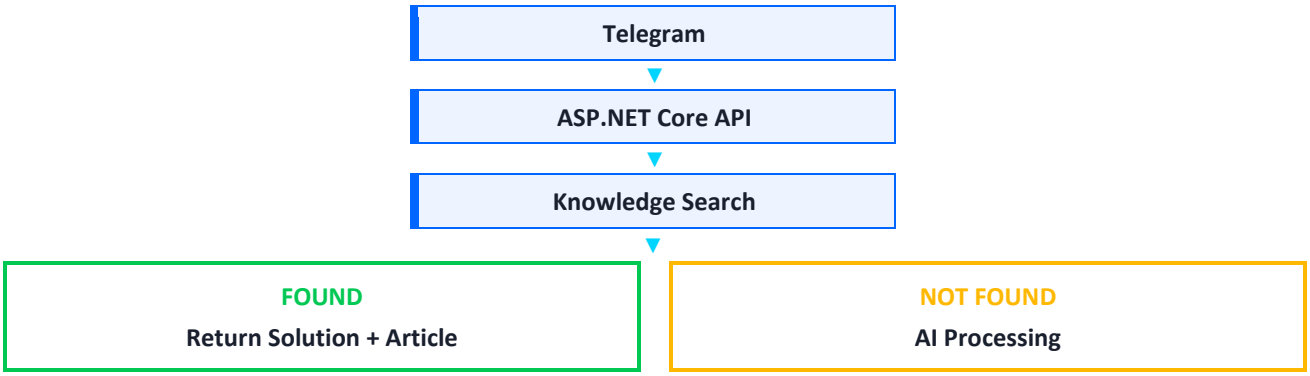
The platform shall provide a Telegram Bot.

A developer can send a technical problem to the bot.

The bot shall:

- Search the existing knowledge base.
- Find similar problems.
- Return relevant solutions.
- Provide the related article link.
- Use the LLM only when existing knowledge is insufficient or when AI assistance is explicitly required.

Possible flow:



5.21 Discord Integration

The system may provide a Discord community integration.

Possible uses:

- Developer community.
- Debugging discussions.
- Technology channels.
- Challenge announcements.

- New knowledge articles.
- Community notifications.
- AI-assisted bot interactions.

> Discord is an optional integration and may be implemented after the core platform is completed.

5.22 Cloudflare Turnstile

Cloudflare Turnstile may be used to protect public endpoints and forms from:

- Automated spam.
- Bot submissions.
- Abuse.
- Excessive public requests.

It may be applied to:

- Registration.
- Public problem submission.
- Contact/support forms.
- Other abuse-sensitive endpoints.

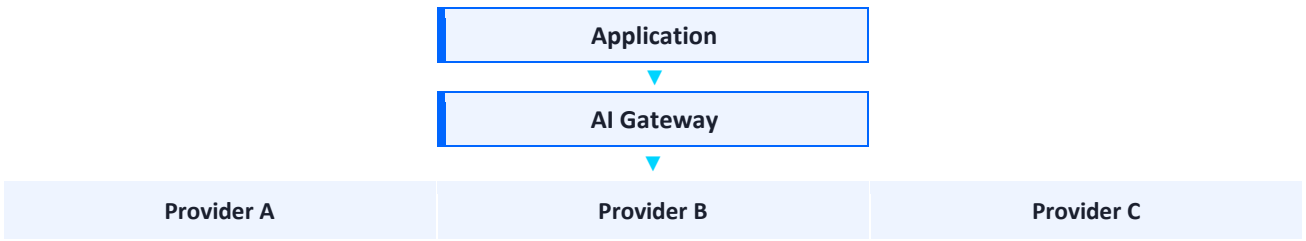
5.23 AI Gateway

The platform shall use an internal AI Gateway rather than directly coupling application logic to a single LLM provider.

The AI Gateway shall provide:

- Provider selection.
- Model selection.
- Request routing.
- Usage tracking.
- Error handling.
- Retry logic.
- Provider failover.
- Token/usage tracking.
- Prompt management.
- Response normalization.

Concept:



5.24 Free LLM Provider Failover

The platform shall support multiple free or free-tier LLM providers.

The system shall maintain provider information such as:

- Provider.
- Model.
- Priority.
- Status.
- Requests used.
- Token usage when available.
- Rate limits when known.
- Last error.
- Cooldown period.
- Availability.

When a provider reaches a rate limit or becomes unavailable:

1. Detect failure.
2. Mark provider temporarily unavailable.
3. Select another available provider.
4. Retry according to configured policy.
5. Return the result to the user without exposing internal provider switching.

> Provider limits shall be treated as dynamic configuration and should not be hard-coded as permanent values.

5.25 AI Prompt Management

The system should maintain structured prompts for different AI operations.

Examples:

Debugging Prompt	**Root Cause Prompt**	**Solution Prompt**
Article Generation Prompt	**Challenge Generation Prompt**	**Challenge Evaluation Prompt**
Code Review Prompt	**Repository Analysis Prompt**	**Solution Synthesis Prompt**

Prompt versions should be tracked where practical.

6. Administrator Requirements

Administrators shall be able to:

- Manage users.
- Manage roles.
- View problems.
- View debugging attempts.
- Review generated articles.
- Manage article status.
- View solution versions.
- View AI usage.
- View provider health.

- Manage AI providers/models.
- View failed AI requests.
- Manage tags/technologies.
- Review reported content.
- Manage platform configuration.

7. Non-Functional Requirements

7.1 Performance

The system should:

- Minimize unnecessary LLM calls.
- Use cached/reusable knowledge when possible.
- Avoid sending unnecessary repository files to LLMs.
- Use pagination for large lists.
- Use asynchronous processing where appropriate.
- Optimize database queries.

7.2 Security

The system shall:

- Use secure authentication.
- Use authorization.
- Protect API endpoints.
- Validate user input.
- Protect GitHub tokens.
- Never expose provider API keys to the frontend.
- Store secrets securely.
- Apply rate limiting where required.
- Protect against common web vulnerabilities.
- Use HTTPS.

7.3 Privacy

The system shall:

- Clearly define how submitted code and logs are stored.
- Avoid exposing private repository data publicly.
- Respect GitHub permissions.
- Restrict private data to authorized users.
- Avoid placing sensitive credentials inside LLM prompts.

7.4 Reliability

The system should:

- Handle LLM provider failures.
- Support provider failover.

- Preserve debugging history.
- Avoid losing successful solutions.
- Log important system errors.

7.5 Maintainability

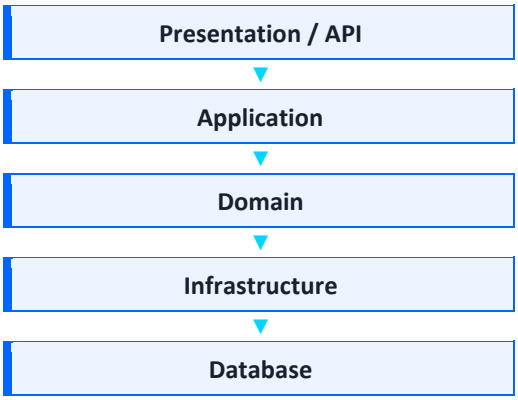
The backend should follow:

- Clean architecture principles where practical.
- SOLID principles.
- Dependency Injection.
- Separation of concerns.
- Repository pattern where justified.
- Unit of Work where justified.
- DTOs.
- Service layer.
- Centralized exception handling.
- Centralized logging.

8. Proposed Backend Architecture

Component	Technology
Frontend	Undefined Yet
Backend	ASP.NET Core Web API

Suggested logical layers:



Suggested modules/services:

Authentication Service	**User Service**	**Problem Service**	**Debugging Service**
Similarity Search Service	**Knowledge Service**	**Article Service**	**Challenge Service**
GitHub Service	**Code Review Service**	**Telegram Service**	**Discord Service**
AI Gateway	**Provider Management Service**	**Usage Tracking Service**	**Notification Service**

Figure 2 on the following page shows how the frontend, backend layers, AI Gateway, database, LLM providers, and external integrations fit together.

Figure 2 — System Architecture

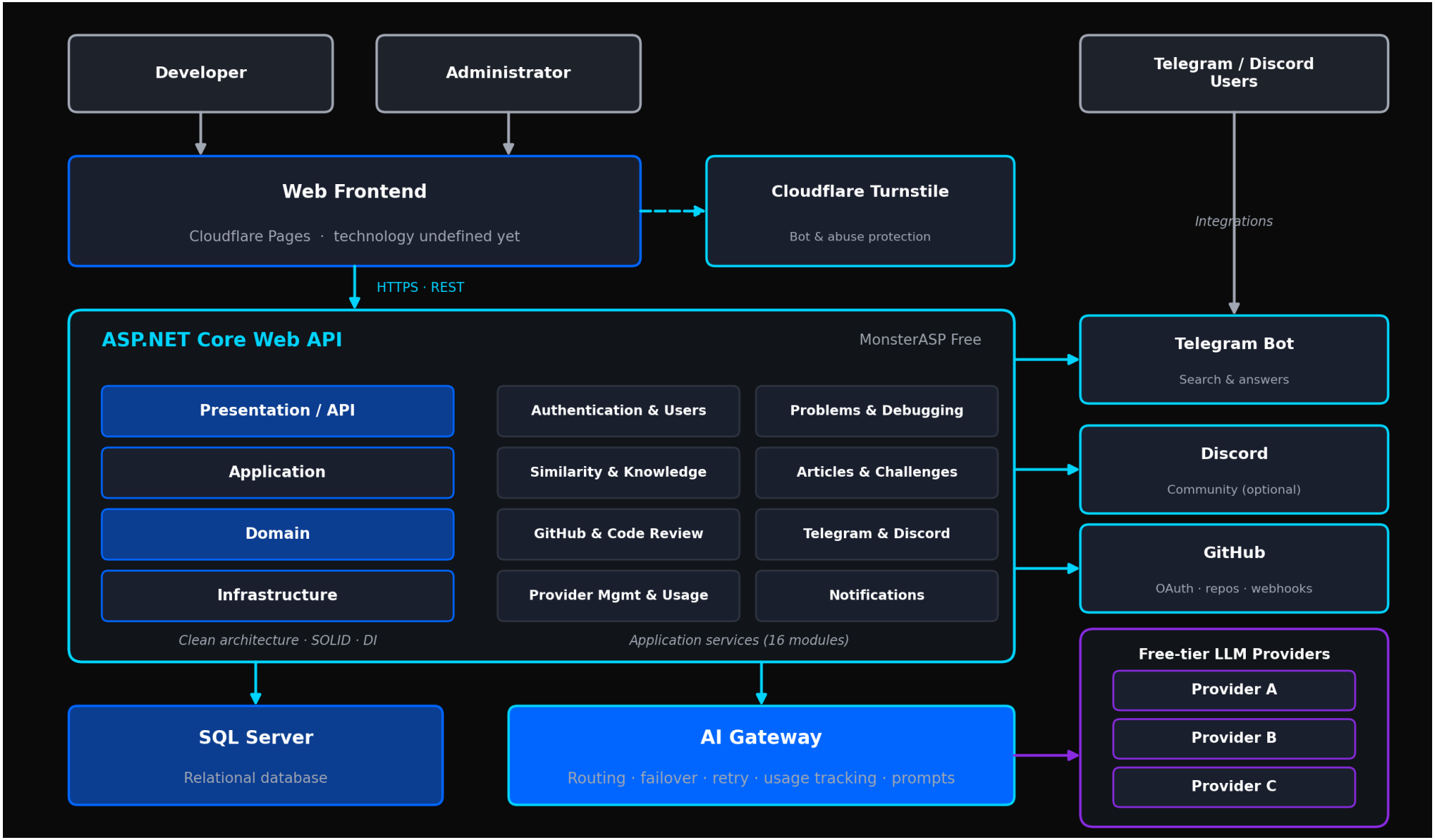


Figure 2. System architecture: web frontend, ASP.NET Core Web API, AI Gateway with LLM providers, SQL Server, and external integrations.

9. Proposed Database Entities

Initial entity list:

Domain Area	Entities
Identity	Users · Roles · UserRoles · DeveloperProfiles
Problems & Debugging	Problems · ProblemRequests · ProblemAttachments · DebuggingSessions · DebuggingAttempts · DeveloperFeedback · Solutions · SolutionVersions · SuccessfulSolutions
Knowledge	KnowledgeArticles · KnowledgeArticleVersions · ArticleProblems
Taxonomy	Technologies · Tags · ProblemTags
Statistics	ProblemStatistics
Challenges	Challenges · ChallengeRequirements · ChallengeAttempts · ChallengeEvaluations
GitHub	GitHubAccounts · GitHubRepositories · GitHubBranches · GitHubFiles · GitHubWebhooks
Code Review	CodeReviews · CodeReviewFindings
AI Infrastructure	AIProviders · AIModels · AIRequests · AIResponses · AIUsageLogs · AIProviderHealth
Search & Similarity	Embeddings · SimilarityMatches
Telegram	TelegramUsers · TelegramMessages
Discord	DiscordUsers · DiscordEvents
Notifications	Notifications
Auditing	AuditLogs

10. Important Data Relationships

- A Developer can have many Problems.
- A Problem can have many Requests.
- A Problem can have many Debugging Attempts.
- A Debugging Attempt can have one or more AI Responses.
- A Problem can have multiple Solutions.
- A Problem can have one current Knowledge Article.
- A Knowledge Article can have multiple versions.
- A Problem can have multiple Challenges.
- A Challenge can have many Attempts.
- A Challenge Attempt can be linked to a GitHub Repository.
- A Challenge Attempt can have an AI Evaluation.
- A Developer can connect one or more GitHub accounts according to implementation rules.
- A GitHub Account can access multiple repositories according to granted permissions.
- A Repository can have multiple Code Reviews.
- A Code Review can contain multiple Findings.

- An AI Provider can have multiple Models.
- An AI Request shall be linked to the selected Provider/Model when applicable.
- Similarity records shall connect new requests with potentially related historical problems.

11. API Modules

Suggested API areas:

`/api/auth`	`/api/users`	`/api/problems`	`/api/debugging`
`/api/solutions`	`/api/knowledge`	`/api/articles`	`/api/challenges`
`/api/github`	`/api/code-review`	`/api/telegram`	`/api/discord`
`/api/ai`	`/api/providers`	`/api/search`	`/api/statistics`
`/api/admin`			

Example endpoints:

Area	Method	Endpoint
Problems	**POST**	`/api/problems`
Problems	**GET**	`/api/problems/{code}`
Problems	**POST**	`/api/problems/{id}/debug`
Problems	**POST**	`/api/problems/{id}/feedback`
Problems	**POST**	`/api/problems/{id}/solve`
Articles	**GET**	`/api/articles`
Articles	**GET**	`/api/articles/{id}`
Challenges	**POST**	`/api/challenges`
Challenges	**POST**	`/api/challenges/{id}/start`
Challenges	**GET**	`/api/challenges/{id}`
Challenges	**POST**	`/api/challenges/{id}/evaluate`
GitHub	**GET**	`/api/github/repositories`
GitHub	**GET**	`/api/github/repositories/{id}/files`
GitHub	**POST**	`/api/github/challenges/{id}/repository`
Code Review	**POST**	`/api/code-review`
Code Review	**GET**	`/api/code-review/{id}`
Telegram	**POST**	`/api/telegram/webhook`

12. AI Processing Pipeline

The recommended AI processing pipeline is:

1	Receive user request.
2	Validate request.
3	Normalize technical information.
4	Extract technology/error indicators.
5	Search similar problems.
6	Evaluate similarity confidence.
7	If suitable knowledge exists, retrieve successful solutions.
8	Build optimized AI context.
9	Send request to AI Gateway.
10	Select available provider/model.
11	Receive AI response.
12	Validate/normalize response.
13	Store request and response.
14	Present solution to developer.
15	Wait for developer feedback.
16	If solved, finalize knowledge.
17	If not solved, continue debugging.

13. Token Optimization Strategy

Token optimization shall be treated as a core system requirement.

Strategies:

- Similarity search before LLM calls.
- Reuse successful historical solutions.
- Send only relevant repository files.
- Summarize large logs before full analysis where appropriate.
- Cache reusable AI outputs.
- Use specialized prompts.
- Limit unnecessary context.
- Track provider usage.
- Fail over between providers.
- Reuse knowledge articles.
- Synthesize multiple successful solutions into improved reusable knowledge.

14. Knowledge Quality Strategy

The system should prioritize solutions that have evidence of success.

Potential solution metadata:

- Number of successful uses.
- Number of failed uses.
- Last successful use.
- Technologies.
- Related problems.
- Developer feedback.
- Article version.
- AI-generated confidence metadata.

The system should not treat popularity alone as proof of correctness.

Knowledge should be improved based on successful outcomes and review mechanisms.

15. Search System

The platform shall support:

- Keyword search.
- Technology filtering.
- Tag filtering.
- Problem code search.
- Error message search.
- Semantic similarity search.

Search results should display:

- Problem title.
- Problem code.
- Short description.
- Technologies.
- Number of reported developers.
- Relevance.

16. Notifications

The system may support notifications for:

- Challenge created.
- Challenge evaluation completed.
- GitHub event received.
- Article updated.
- New related knowledge.
- System notifications.

Notification channels may include:

- In-app.
- Telegram.
- Discord.

17. Logging and Auditing

The system should log:

- Authentication events.
- Important API requests.
- AI requests.
- AI provider failures.
- GitHub integration events.
- Challenge events.
- Administrative actions.
- Security-related events.

Audit logs should avoid storing unnecessary sensitive information.

18. Error Handling

The API shall use centralized error handling.

Errors should provide:

- HTTP status.
- Application error code.
- Safe user-facing message.
- Correlation/request identifier where useful.

Internal exceptions and secrets shall not be exposed to users.

19. MVP Scope

PHASE 1 Core MVP	• Developer Authentication. • Problem Submission. • AI Debugging. • Step-by-Step Solution. • Solved / Still Have a Problem workflow. • Debugging History. • Database storage. • Public Knowledge Base. • Educational Article generation. • Problem search. • Basic similarity search. • AI Gateway. • Multiple LLM provider failover. • Basic token/usage tracking.
PHASE 2	• GitHub OAuth. • GitHub Repository Analysis. • GitHub-based Debugging. • AI Code Review. • GitHub Webhooks.
PHASE 3	• AI Challenge Generation. • GitHub Challenge Repository Creation. • Challenge Submission. • AI Challenge Evaluation.
PHASE 4	• Telegram Bot. • Discord Integration. • Advanced semantic search. • Solution synthesis. • Automatic article evolution. • Advanced statistics and analytics.

20. Out of Scope for Initial Version

The initial version should not require:

- Desktop EXE.
- Local Windows Agent.
- Ollama installation on the developer device.
- Direct operating system control.
- Remote execution of arbitrary commands on the developer's machine.

The platform is Web-based.

21. Deployment

Layer	Technology	Hosting
Frontend	Undefined Yet	Cloudflare Pages
Backend	ASP.NET Core Web API	MonsterASP Free
Database	SQL Server	—
AI	Multiple Free / Free-Tier LLM APIs through the internal AI Gateway	—

External Integrations:

GitHub	Telegram	Discord	Cloudflare Turnstile

22. Security Principles for GitHub

GitHub integration shall follow the principle of least privilege.

The system shall:

- Request only required permissions.
- Securely store access credentials/tokens.
- Never expose tokens to the frontend.
- Avoid storing unnecessary repository content.
- Avoid exposing private repository information.
- Validate webhook authenticity.
- Restrict repository access according to user permissions.

23. Future Enhancements

Possible future features:

- Pull Request automatic review.
- AI-generated test cases.
- Automated bug reproduction where safe.
- Advanced repository architecture mapping.
- Developer skill analytics.
- Personalized learning paths.
- Team knowledge bases.
- Team code review analytics.
- Advanced challenge difficulty adaptation.
- Multi-language programming support.
- More AI providers.
- More communication integrations.
- Knowledge moderation workflow.
- Community voting and feedback.
- Advanced reputation system.

24. Project Differentiator

The platform is not intended to be only:

- An AI chatbot.
- A simple debugging assistant.
- A Stack Overflow clone.
- A code review tool.
- An e-learning platform.

The core concept is combining these capabilities into one continuous developer workflow:



The system therefore creates a continuously improving developer knowledge and practice ecosystem based on real-world problems.

25. Success Criteria

The project shall be considered successful when a developer can:

1. Submit a real programming problem.
2. Receive AI debugging assistance.
3. Understand the root cause.
4. Follow step-by-step instructions.
5. Confirm whether the problem was solved.
6. Continue debugging if it was not solved.
7. Have the successful problem stored.
8. Find the problem later through the knowledge base.
9. Read an educational article generated from the problem.
10. Start a related practical challenge.
11. Receive a GitHub repository for the challenge.
12. Submit a solution.
13. Receive AI evaluation.
14. Connect GitHub repositories for debugging/code review.
15. Search similar historical problems.
16. Benefit from previously successful solutions.
17. Use Telegram to find existing solutions.
18. Experience provider failover without manually selecting an LLM provider.

END OF SOFTWARE REQUIREMENTS SPECIFICATION